

JavaScript - Day 2

Strings, Arrays, Objects & Memory Concepts

Student Notes & Interview Preparation Guide

Topics covered: Strings & String Methods | Arrays & Array Methods | Objects & Object Methods | Dot vs Bracket Notation | Arrays vs Objects | String Immutability | Pass by Value vs Pass by Reference | Mutation Bugs | Shallow Copy vs Deep Copy

Learning Objectives

By the end of this chapter, students will be able to:

- Understand Strings and String Methods
- Understand Arrays and Array Methods
- Understand Objects and Object Methods
- Access object properties using Dot and Bracket Notation
- Differentiate Arrays and Objects
- Understand String Immutability
- Understand Pass by Value and Pass by Reference
- Understand Mutation Bugs
- Understand Shallow Copy and Deep Copy

1. Strings

What is a String?

A String is a sequence of characters enclosed inside quotes.

```
let name = "Yatheswar";  
let city = 'Chennai';
```

Strings are used to store:

- Names
- Emails
- Passwords
- Addresses
- Messages

Real World Example

Every character in a string has an index, starting from 0.

```
H e l l o  
0 1 2 3 4
```

String Length

Returns the total number of characters in a string.

```
let str = "JavaScript";  
  
console.log(str.length);
```

Output:

```
10
```

charAt()

Returns the character at a given position.

```
let str = "JavaScript";  
  
console.log(str.charAt(4));
```

Output:

```
S
```

String Concatenation

Joining two or more strings together.

Using +

```
let firstName = "Yathe";  
let lastName = "Eswar";  
  
console.log(firstName + " " + lastName);
```

Using concat()

```
console.log(  
  firstName.concat(" ", lastName)  
);
```

Output:

```
Yathe Eswar
```

Extracting Parts of a String

slice()

Extracts a part of a string.

```
let str = "JavaScript";  
  
console.log(str.slice(0,4));
```

Output:

```
Java
```

Negative Index

```
console.log(str.slice(-6));
```

Output:

```
Script
```

substring()

```
let str = "JavaScript";  
  
console.log(str.substring(4,10));
```

Output:

```
Script
```

substr()

```
let str = "JavaScript";  
  
console.log(str.substr(4,6));
```

Output:

Script

Note: *substr()* is considered legacy and should generally be avoided in modern JavaScript.

Case Conversion

toUpperCase()

```
let str = "hello";  
  
console.log(str.toUpperCase());
```

Output:

HELLO

toLowerCase()

```
let str = "HELLO";  
  
console.log(str.toLowerCase());
```

Output:

hello

Trimming Strings

Used to remove unnecessary spaces.

```
let str = "  Hello World  ";  
  
console.log(str.trim());
```

Output:

Hello World

Other Methods

```
trimStart()
```

```
trimEnd()
```

Real World Use

Used in:

- Login Forms
- Registration Forms
- Search Bars

Replacing Text

replace()

Replaces the first occurrence.

```
let str = "Dogs are cute";

console.log(
  str.replace("Dogs", "Cats")
);
```

replaceAll()

Replaces all occurrences.

```
let str =
"Dogs are cute. Dogs are friendly";

console.log(
  str.replaceAll("Dogs", "Cats")
);
```

Search Methods

includes()

```
let str = "My Portfolio is Ready";

console.log(
  str.includes("Portfolio")
);
```

Output:

```
true
```

indexOf()

```
console.log(  
  str.indexOf("Portfolio")  
);
```

Returns the index position.

search()

```
console.log(  
  str.search("Portfolio")  
);
```

match()

```
console.log(  
  str.match("Ready")  
);
```

Returns the matched value.

String Immutability

What is Immutability?

Immutable means: cannot be changed after creation. Strings in JavaScript are immutable.

Example

```
let str = "Hello";  
  
str[0] = "Y";  
  
console.log(str);
```

Output:

```
Hello
```

No change occurs.

Why?

Whenever JavaScript appears to modify a string, it actually creates a new string.

```
let str = "Hello";  
  
str = str + " World";
```

Memory Concept

```
Old String  -> "Hello"

New String  -> "Hello World"
```

Important Point

Methods like:

```
toUpperCase()
replace()
trim()
slice()
concat()
```

return new strings. They do NOT modify the original string.

Interview Question

Q: Are Strings Mutable or Immutable?

Answer: Strings are immutable. Any modification creates a new string.

2. Arrays

What is an Array?

An Array is a collection of multiple values stored inside a single variable.

```
let fruits = [
  "Apple",
  "Banana",
  "Grapes"
];
```

Why Arrays?

Without arrays:

```
let student1 = "Rahul";
let student2 = "Harsha";
let student3 = "Akash";
```

With arrays:

```
let students =
["Rahul", "Harsha", "Akash"];
```

Much easier to manage.

Characteristics of Arrays

- Ordered
- Indexed
- Mutable
- Dynamic
- Heterogeneous

```
let data = [  
  "Yathe",  
  25,  
  true  
];
```

Array Memory Concept

```
let x = [1,2,3];  
let y = x;  
  
console.log(x === y);
```

Output:

```
true
```

Because both variables point to the same memory location.

Array Methods

length

```
fruits.length
```

at()

```
fruits.at(1)
```

push()

Adds an element at the end.

```
fruits.push("Mango");
```

pop()

Removes the last element.

```
fruits.pop();
```

unshift()

Adds an element at the beginning.

```
fruits.unshift("Orange");
```

shift()

Removes the first element.

```
fruits.shift();
```

concat()

Merges arrays.

```
let arr3 = arr1.concat(arr2);
```

slice()

Creates a copy of a selected portion.

```
arr.slice(1,4);
```

splice()

Adds or removes elements.

```
arr.splice(2,1,"New");
```

Search Methods (Arrays)

indexOf()

Returns the first occurrence.

```
arr.indexOf("Coolie");
```

lastIndexOf()

Returns the last occurrence.

```
arr.lastIndexOf("Coolie");
```

includes()

```
arr.includes("OG");
```

Returns true or false.

String ↔ Array Conversion

split()

Converts a String into an Array.

```
let str = "JS is awesome";

console.log(
  str.split(" ")
);
```

join()

Converts an Array into a String.

```
let arr = [
  "JS",
  "is",
  "awesome"
];

console.log(
  arr.join(" ")
);
```

Output:

```
JS is awesome
```

Common Use Cases

Convert CSV Data

```
let csv = "a,b,c,d";

let arr = csv.split(",");
```

Remove Extra Spaces

```
let text =
  "  JS    is    fun  ";
```

```
let clean =  
text.trim()  
  .split(/\s+/)  
  .join(" ");
```

Reverse a String

```
let str = "JavaScript";  
  
let reversed =  
str.split("")  
  .reverse()  
  .join("");
```

Arrays vs Objects

Introduction

Both Arrays and Objects store collections of data. However, they are designed for different purposes.

Structure

Array

Stores data using numeric indexes.

```
let fruits = [  
  "Apple",  
  "Banana",  
  "Grapes"  
];
```

Object

Stores data using key-value pairs.

```
let student = {  
  name: "Rahul",  
  age: 22  
};
```

Comparison Table

Feature	Array	Object
Storage	Indexed	Key-Value
Order Matters	Yes	Usually No

Feature	Array	Object
Access	Index	Key
Best For	Lists	Entities
Built-in Methods	Many	Few

When to Use Arrays?

Use arrays when:

- Order matters
- Data behaves like a list
- Frequent additions/removals occur

Examples:

- Products
- Movies
- Messages
- Students

When to Use Objects?

Use objects when:

- Data represents a single entity
- Data has attributes

Examples:

- User
- Employee
- Product
- Configuration

3. Objects

What is an Object?

Objects store data in key-value pairs.

```
let student = {  
  name: "Yathe",  
  age: 25,  
  city: "Chennai"  
};
```

Creating Objects

Method 1: Object Literal

```
let obj = {  
  name: "Yathe"  
};
```

Method 2: Empty Object

```
let obj = {};  
  
obj.name = "Yathe";
```

Method 3: Using new Object()

```
let obj = new Object();
```

Accessing Properties

JavaScript provides two methods:

1. Dot Notation
2. Bracket Notation

Dot Notation

```
student.name
```

Simple and readable.

Bracket Notation

```
student["name"]
```

More flexible.

Dot Notation vs Bracket Notation

Feature	Dot	Bracket
Easy to Read	Yes	No
Variable Names	No	Yes
Special Characters	No	Yes
Spaces Allowed	No	Yes

Dynamic Property Access

```
let key = "name";

console.log(
  student[key]
);
```

Real World Example

```
let selectedProperty = "email";

console.log(
  user[selectedProperty]
);
```

Used when property names are determined at runtime.

Add, Modify and Delete Properties

Add Properties

```
student.college =
  "Veltech";
```

Modify Properties

```
student.age = 26;
```

Delete Properties

```
delete student.age;
```

Object Methods

Functions inside objects are called methods.

```
let person = {

  firstName:"Yathe",

  lastName:"Eswar",

  fullName:function(){

    return this.firstName +
      " " +
```

```
        this.lastName;  
    }  
};
```

Nested Objects

Objects inside objects.

```
let student = {  
    name:"Krishna",  
    friend:{  
        name:"Lokesh",  
        age:21  
    }  
};
```

Access:

```
student.friend.name
```

Useful Object Methods

Object.keys()

Returns keys.

```
Object.keys(student);
```

Object.values()

Returns values.

```
Object.values(student);
```

hasOwnProperty()

Checks whether a property exists.

```
student.hasOwnProperty("name");
```

Output:

```
true
```

Important Note About Object Keys

JavaScript converts keys into strings.

```
let obj = {  
  21: "Age",  
  true: "Value"  
};
```

Internally:

```
{  
  "21": "Age",  
  "true": "Value"  
}
```

Pass By Value vs Pass By Reference

Note: One of the most important JavaScript interview topics.

Pass By Value

Used by primitive datatypes:

- Number
- String
- Boolean
- Undefined
- Null
- Symbol
- BigInt

Example

```
let a = 10;  
  
let b = a;  
  
b = 20;
```

Output:

```
a = 10  
b = 20
```

Memory Diagram

```
a → 10
```

```
b → 10
```

Separate copies.

Pass By Reference

Used by:

- Objects
- Arrays
- Functions

Example

```
let obj1 = {
  value:10
};

let obj2 = obj1;
```

Memory:

```
obj1  ┌───┐
      │   │ → {value:10}
obj2  └───┘
```

Result

```
obj2.value = 20;
```

Now:

```
obj1.value
```

Output:

```
20
```

Comparison Table

Feature	Pass By Value	Pass By Reference
Memory	Copy Created	Reference Shared
Primitive Types	Yes	No
Objects & Arrays	No	Yes
Original Changes	No	Yes

Mutation Bug

```
let user = {  
  name: "Yathe"  
};  
  
function change(obj){  
  obj.name = "Changed";  
}  
  
change(user);
```

Output:

```
{name: "Changed"}
```

The original object changes unexpectedly.

Fix Using Spread Operator

```
let copy = {  
  ...original  
};
```

Creates a shallow copy.

Shallow Copy

Copies only top-level properties.

```
let copy = {  
  ...original  
};
```

Nested objects still share memory.

Deep Copy

Creates a completely independent copy.

```
let copy =  
JSON.parse(  
  JSON.stringify(original)  
);
```

Comparison

Feature	Shallow Copy	Deep Copy
Top Level Copied	Yes	Yes
Nested Objects Copied	No	Yes
Shared References	Yes	No
Independent Copy	No	Yes

Key Takeaways

- Strings are immutable.
- Arrays are mutable and index-based.
- Objects store key-value pairs.
- Dot notation is readable.
- Bracket notation is flexible.
- Arrays are best for lists.
- Objects are best for entities.
- Primitive types follow pass by value.
- Objects and arrays share references.
- Spread operator creates shallow copies.
- Deep copy creates fully independent objects.

Interview Questions

Basic

3. What is a String?
4. What is an Array?
5. What is an Object?
6. What is String Immutability?
7. What is Object Mutability?

Intermediate

8. Difference between slice() and splice()?
9. Difference between split() and join()?
10. Difference between Arrays and Objects?

11. Difference between Dot and Bracket Notation?

12. Difference between Pass By Value and Pass By Reference?

Advanced

13. Why are Strings immutable?

14. What is a Mutation Bug?

15. What is Shallow Copy?

16. What is Deep Copy?

17. Why does changing one object affect another?

Practice Assignment

Create a Student Management System.

```
{  
  name:"",  
  age:"",  
  course:"",  
  marks:[]  
}
```

Implement:

- Add Student
- Update Student
- Delete Student
- Search Student
- Display All Students